Reviewer Pack — Dual Matroid Rank Bounded by Karchmer-Wigderson Protocol Cos…

Entry 812ab278de1e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 20:00:20 UTC

Dual Matroid Rank Bounded by Karchmer-Wigderson Protocol Cost

Entry ID: `812ab278de1e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 20:00:20 UTC

1. Conjecture

Field A (mathematical branch): Matroid Duality Theory **Field B** (complexity object): Karchmer-Wigderson Communication Complexity

Statement:

> For any Boolean function f, the minimal Karchmer-Wigderson protocol cost C(f) satisfies C(f) ≥ rank(M_dual(f)) where M_dual(f) is the dual matroid of the incidence structure of f's clauses. Equality holds for all monotone Boolean functions.

Rationale (proposer's reasoning):

> Matroid duality captures complementary structural dependencies, while Karchmer-Wigderson protocols model communication for circuit-depth lower bounds. Their direct linkage could reveal hidden constraints on protocol efficiency via matroid rank, bypassing traditional algebraic barriers.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `71789124ea1b0754`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(cols):
            if j != i:
                factor = -matrix[j][i] / matrix[i][i]
                for k in range(rows):
                    matrix[j][k] += factor * matrix[i][k]
    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    row echelon_form = gaussian_elimination(matrix)
    rank = 0
    for i in range(min(rows, cols)):
        if row_echelon_form[i][i] != 0:
            rank += 1
    return rank

def incidence_matroid_rank(clauses):
    n = len(clauses[0])
    matroid = {var: set() for var in range(n)}
    for i, clause in enumerate(clauses):
        for var in clause:
            matroid[var].add(i)

    def is_independent(set_of_clauses):
        independent_set = list(set_of_clauses)
        if len(independent_set) == 0:
            return True
        if len(independent_set) > n:
            return False
        for _ in range(n - len(independent_set)):
            new_clause = set(range(n)) - matroid[sum(independent_set, start=0)]
```

```
                independent_set.append(new_clause)
            return len(independent_set) == n

        max_rank = 0
        for i in range(1 << n):
            set_of_clauses = [j for j in range(n) if (i & (1 << j)) != 0]
            if is_independent(set_of_clauses):
                max_rank = max(max_rank, len(set_of_clauses))

        return max_rank

    def karchmer_wigderson_cost(clauses):
        n = len(clauses[0])
        matroid_rank = incidence_matroid_rank(clauses)
        return math.ceil(math.log2(matroid_rank + 1))

    def generate_random_3cnf(n, m):
        variables = list(range(n))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 3)
            clauses.append(clause)
        return clauses

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        n = random.randint(5, 40)
        m = random.randint(n * 2, n * 5)
        clauses = generate_random_3cnf(n, m)

        matroid_rank = incidence_matroid_rank(clauses)
        karchmer_wigderson_cost_value = karchmer_wigderson_cost(clauses)

        return {
            "metric_name": "Karchmer-Wigderson cost",
            "metric_value": karchmer_wigderson_cost_value,
            "instances_tested": 1,
            "conjecture_holds": karchmer_wigderson_cost_value >= matroid_rank,
            "counterexample": "" if karchmer_wigderson_cost_value >= matroid_rank else f"Graph with n=
        }

    if __name__ == "__main__":
        import sys
        seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
        results = []

        for seed in seeds:
            result = run_trial(seed)
            print(f"TRIAL: {result}")
            results.append(result)

        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
```

```
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with n={results[0]['instances_tested']},
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_482a9eeb.py", line 81, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_482a9eeb.py", line 65, in run_trial
    matroid_rank = incidence_matroid_rank(clauses)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_482a9eeb.py", line 41, in incidence_mat
    if all(len(matroid[var] & independent_set) == 1 for _ in independent_set):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_482a9eeb.py", line 41, in <genexpr>
    if all(len(matroid[var] & independent_set) == 1 for _ in independent_set):
               ~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~
TypeError: unsupported operand type(s) for &: 'set' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError in matroid set operations, preventing data collection | next: Fix the set/list intersection bug in incidence_matroid_rank function

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112276
2	preregistration	ollama_remote	qwen3:8b	0	0	31611
3	novelty	ollama_remote	qwen3:8b	0	0	27392
4	novelty	ollama_remote	qwen3:8b	0	0	16601
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13095

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9667
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9137
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12280
9	judge	ollama_remote	qwen3:8b	0	0	27168

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 259227 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/812ab278de1e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/812ab278de1e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/812ab278de1e.tar.gz` (if generated)